
COMP2511

Tutorial 5

Last Week's Tutorial

- Design Principles (SOLID and Law of Demeter)
- Design by Contract
- Strategy Pattern
- Streams

This Week's Tutorial

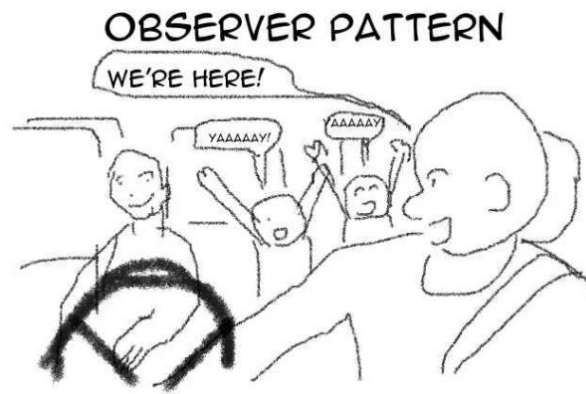
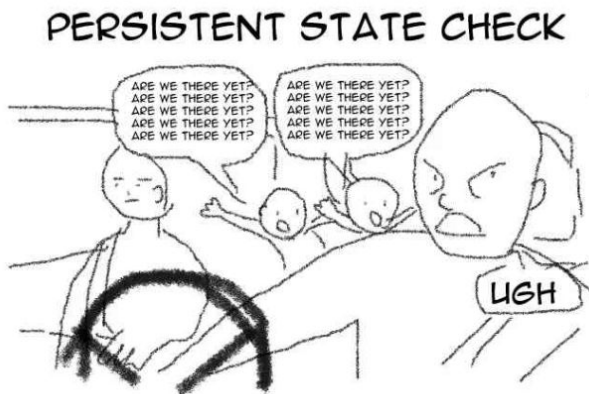
- More Design Patterns
 - Observer Pattern
 - Composite Pattern
 - Factory Pattern

Observer Pattern

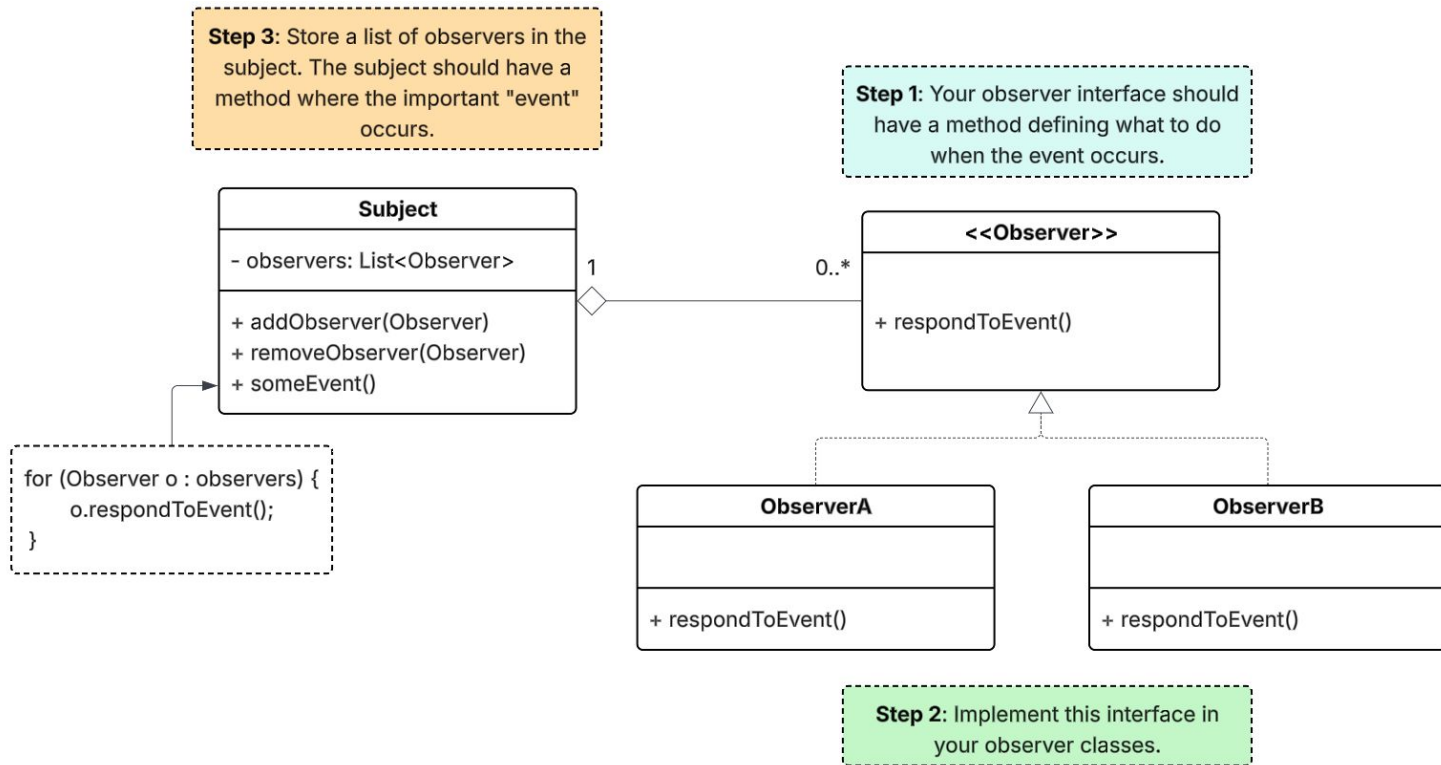
- If you were subscribed to a YouTube channel and wanted to watch that channel's videos as soon as they came out, would you rather:
 - Keep refreshing the YouTube channel every 5 seconds?
 - Receive a notification whenever a new video has been uploaded?
- The latter case is more sensible - when a channel publishes a new video, its subscribers get notified.
- We can frame this scenario in a particular way: There is an important **event** (a new video being uploaded) that some people are waiting on. As soon as that event happens, everyone waiting will get **notified** and respond to it; this is the idea of the Observer Pattern.

Observer Pattern

- The Observer Pattern is a design pattern where a subject maintains a list of observers, who are **notified** once an important event happens with the subject.



Observer Pattern: Implementation

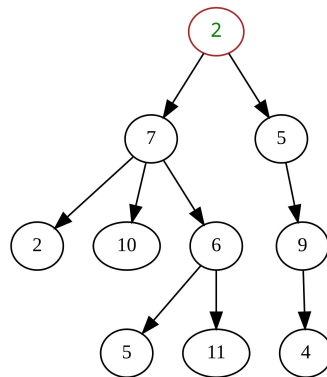


Live Example: Observer Pattern

- In **src/youtube**, create a model for the following requirements of a YouTube-like video creating and watching service using the Observer Pattern.
 - A YouTuber has a name, subscribers and videos.
 - A User has a name and can subscribe to any YouTuber.
 - When a YouTuber posts a new video, the video gets added to the 'Watch Later' lists of all Users subscribed to that YouTuber.
 - Write a simple test with print statements inside YouTube.java.

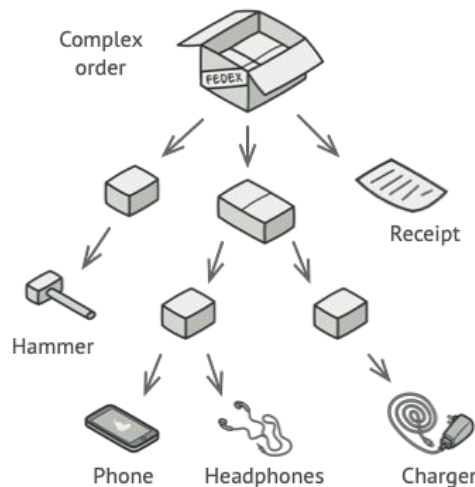
Composite Pattern

- The Composite pattern is a design pattern that organises objects into **tree structures**, i.e. recursive structures where each object is a node connected to zero or more subtrees (a tree can be null/empty).
- Terminology:
 - Nodes that have (non-empty) subtrees as children are referred to as **composites**.
 - Nodes that have no children are referred to as **leaves**.



Composite Pattern

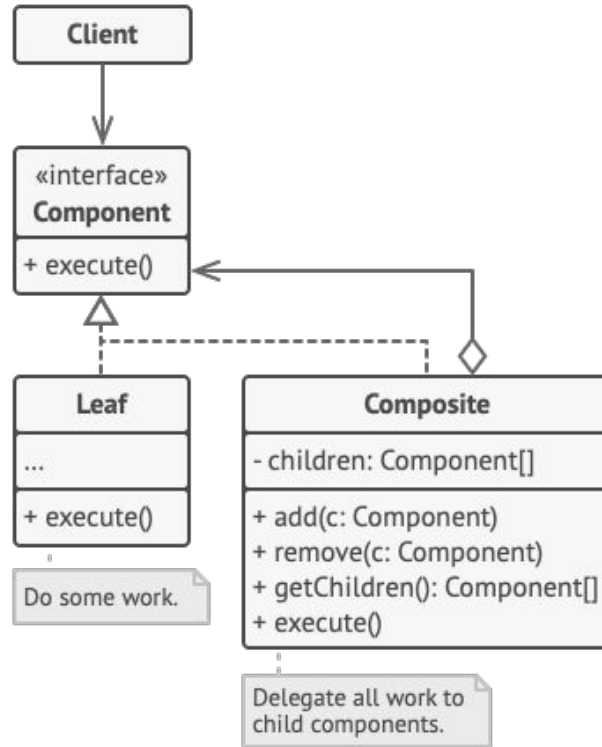
- Imagine you have a box of items, and you're trying to count the number of items inside of the box. Inside that box, you could have individual items and more boxes, which in turn could have more items and boxes.



Composite Pattern

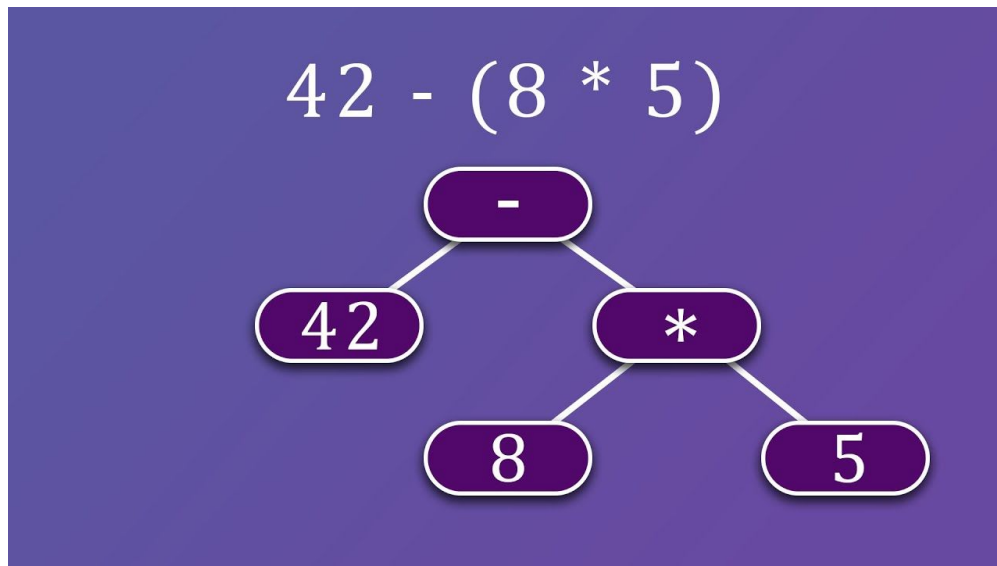
- This can be viewed in terms of a tree: boxes are **composite** objects, while the individual items are **leaves**.
 - The number of items in a box is however many items are inside the box *plus* the number of items inside any smaller boxes (the **recursive case**).
 - Everything else is considered as a single item (the **base case**).
- The aim of the Composite Pattern is to be able to interact with composite and leaf objects in the same manner. In OOP terms, this means that composites and leaves should implement a common **interface**.
 - In this example, we would want to treat single items and boxes as the same (for the purposes of counting). This would be like if someone already looked through all of the boxes and put a sticky note saying how many items each box had, so counting items becomes easy.

Composite Pattern Implementation



Live Example: Composite Pattern

- How can arithmetic expressions (e.g. 5 , $5 + 7$, $7 * 5 - 6$) be represented in terms of a tree? Which nodes are composites and which are leaves?



Live Example: Composite Pattern

- Inside **src/calculator**, use the Composite Pattern to write a simple calculator that evaluates an expression. Your calculator should be able to **add**, **subtract**, **divide** and **multiply** two expressions.

Factory Pattern

- The Factory Pattern is a design pattern that abstracts away the logic of using and calling constructors to other classes.
- The construction logic is delegated to classes called **factories**. Rather than calling constructors directly, objects get created by calling methods defined in factories, which return objects of the required type.
- This is useful for situations where the logic required to construct an object is complicated and bloats the main program logic.

Factory Pattern

- **Disclaimer:** In this course, the main use of the Factory Pattern is moving complicated construction logic away from the 'important' classes into a factory class. This is a pretty simple definition, and is **different to how you may see the Factory Pattern defined in other resources.**

Live Example: Factory Pattern

- Inside **src/myunsw**, there is a simple system modelling the enrolment of new students into UNSW.
 - To enrol a student, their ID needs to be generated, their program needs to be turned from its string representation to its numeric code (e.g. Computer Science → 3778) and a random value representing their luck must be generated.
- **Refactor the code to use the Factory Pattern to abstract the Student creation logic away from the MyUNSW class.**